```
public class IndexModel: PageModel
{
    [BindProperty]                              ❶
    public string Category { get; set; }        ❶

    [BindProperty(SupportsGet = true)]          ❷
    public string Username { get; set; }        ❷

    public void OnGet()
    {
    }

    public void OnPost(ProductModel model)      ❸
    {
    }
}
```

❶ Properties decorated with [BindProperty] take part in model binding.

❷ Properties are not model-bound for GET requests unless you use SupportsGet.

❸ Parameters to page handlers are also model-bound when that handler is selected.

As described in chapter 15 and shown in the preceding listing, `PageModel` properties are *not* model-bound for `GET` requests, even if you add the `[BindProperty]` attribute. For security reasons, only requests using verbs like `POST` and `PUT` are bound. If you *do* want to bind `GET` requests, you can set the `SupportsGet` property on the `[BindProperty]` attribute to opt in to model binding.

Which part is the binding model?

Listing 16.1 shows a Razor Page that uses multiple binding models: the `Category` property, the `Username` property, and the `ProductModel` parameter (in the `OnPost` handler) are all model-bound.

Using multiple models in this way is fine, but I prefer to use an approach that keeps all the model binding in a single, nested class, which I often call `InputModel`. With this approach, the Razor Page in listing 16.1 could be written as follows: